COMPANION TO CDA V1.1

K8s Event Triage Platform

Project Status — Implementation Snapshot

Owner

David Gardiner · Platform Engineering

Version

1.0 — Draft

Date

2026-04-28

Companion to

CDA v1.1

AGENDA

What we'll cover

01

Executive Snapshot

State of every component

02

System Design

Topology · routing · skills · HITL

03

Component Status

HITL · A2A · LGTM · agentgateway

04

Architecture Decisions

14 key calls + rationale

05

Agent Cards

~20 agents · 5 grouped patterns

SECTION 01

Executive Snapshot

Where every moving part is today

STATE OF EVERY COMPONENT

What's built · what's designed · what's pending

THEME	STATE	DETAIL
Event ingestion (Alloy → Event Hub → Argo Events)	PRODUCTION	Three-tier pipeline tested e2e on `proxmox-k8s`
Argo Workflows triage DAG	PRODUCTION	parse-otlp → fan-out → KAgent A2A → GitLab + Mattermost
KAgent A2A protocol	PRODUCTION	JSON-RPC 2.0 · `message/send` · agent-as-tool proven
Specialist agent roster	PHASE 2	11 namespace agents on worker bundle · ~20 planned
agentgateway (replacing LiteLLM)	PHASE 1 — PARALLEL	Deployed alongside · UAMI working · per-agent migration
Managed LGTM integration	DESIGNED	`managed-lgtm-integration/` · pending platform-team Q&A
Skills (kagent 0.8.0+ git/OCI bundles)	IMPLEMENTED, SPARSE	Mechanism deployed · library grows via learning loop
Memory (pgvector backend)	BLOCKED	Reverted in kagent v0.8.3 · awaiting re-enable
HITL (Teams approval gate)	PRODUCTION	Istio VS+AuthZ live · mock bot smoke test in repo
GitLab ticketing	PRODUCTION	Personal project today · dedicated project + round-robin in flight
ServiceNow / ADO integrations	DESIGNED	MCP servers · not yet implemented

HEADLINE NUMBERS

The shape of the thing

AGENTS

~20

Specialist agents · 11 deployed · 9 planned

TRIAGE TIME

~60s

Event → diagnosis · vs. 30–60 min manual

TIERS

3

critical · warnings · infra · per consumer group

DEDUP LAYERS

3

Alloy · Sensor · Script TTL

Goal state: zero unresolved events — every K8s warning either auto-resolved by an agent or assigned via round-robin to a human within minutes.

SECTION 02

System Design

Topology · routing · skills · output

MGMT CLUSTER ORCHESTRATING WORKERS

Brain in the centre, eyes everywhere

```
┌─────────────────── MANAGEMENT CLUSTER ────────────────────┐
│ Argo Events → Workflows → kagent system agents            │
│   parse-otlp → fan-out → A2A → HITL → ticket + notify     │
│                                                            │
│        agentgateway (LLM + MCP + A2A gateway)             │
│        AKS-MCP (UAMI cross-cluster kubectl)               │
│                        │                                   │
│        Azure Event Hub topics: k8s-events · alerts        │
└────────────────────┬──────────────────────────────────────┘
                     │ Kafka/TLS · SAS Send-only
                     │
┌───────────── WORKER CLUSTERS (1..N) ──────────────────────┐
│ Alloy → Event Hub + managed Mimir/Loki/Tempo              │
│ Worker-local kagent specialists (cert-manager, kyverno…)  │
│ App + system namespaces                                    │
└────────────────────────────────────────────────────────────┘
```

Cross-cluster reads: AKS-MCP via UAMI · workload-identity-bound to a per-cluster Federated Identity Credential.

WHY SPLIT MGMT VS WORKER

Different concerns, different blast radius

CONCERN	MANAGEMENT CLUSTER	WORKER CLUSTER
Cross-cluster orchestration	Yes — AKS-MCP, multi-cluster routing	No — only sees self
System namespaces of interest	`argo`, `argo-events`, `kagent`, `agentgateway-system`, `aks-mcp`	`cert-manager`, `kyverno`, `kro`, `external-secrets`, `monitoring`, `istio-system`, `flux-system`
Triage location	Aggregates events from N workers	Local triage when EventHub round-trip is unnecessary
LLM access	Yes (via agentgateway)	Yes (via agentgateway, mgmt or local)
Latency	Slower — round-trip via Event Hub	Faster — local A2A

Worker-local triage shortens loops from seconds to milliseconds for namespaces that don't need cross-cluster context (cert-manager, kyverno, kro etc.).

NAMESPACE → AGENT ROUTING

Two ConfigMaps drive the dispatch

```
Event arrives ─▶️ parse-otlp extracts: namespace, reason
                        │
                        ▼
                 Reason match? (Evicted, NodeNotReady, FailedMount, …)
                   YES → reason-route agent
                   NO  → namespace-route agent
                           │
                           └── No match → sre-triage-agent (default)
                        │
                        ▼
                 Workflow calls A2A: POST /api/a2a/kagent/<agent>/
                        │
                        ▼
                 Agent loads:
                   • System prompt (in CRD)
                   • Skills (mounted /skills via skills-init)
                   • Memory (load_memory · pgvector — when re-enabled)
                   • Tools (kagent-tool-server via MCP)
                        │
                        ▼
                 Diagnosis JSON returned to workflow
```

Routing config: `eventhub-otlp-pipeline/tier-critical/agent-routing.yaml`

SKILLS · CONTEXT · MEMORY

How agents pick up domain knowledge

LAYER	SOURCE	LOADED WHEN	MECHANISM
System prompt	`Agent.spec.declarative.systemMessage`	Pod start	Inline in CRD
A2A skill metadata	`a2aConfig.skills`	Pod start	Discovery / catalog only — not executed
Executable skills	`spec.skills.gitRefs` or OCI	Pod start	`skills-init` initContainer clones to `emptyDir` at `/skills`
Tools	`spec.declarative.tools` (MCP)	Per-request	RemoteMCPServer or Service · agentgateway can be central MCP gateway
Memory	`spec.declarative.memory` (when re-enabled)	Per-request	`load_memory(query)` · `save_memory(text)` · pgvector
Prompt template data	`promptTemplate.dataSources`	Per-request	ConfigMap-backed builtins + per-namespace overrides

Critical naming rule: `spec.skills` (executable bundles) ≠ `a2aConfig.skills` (metadata only).

PER-REQUEST TRIAGE FLOW

What happens when an agent is invoked

```
                 Workflow sends A2A message → agent pod
                              │
                  ┌───────────┼────────────┐
                  ▼           ▼            ▼
           Static context   Memory       Tools (MCP)
             • prompt        lookup        k8s_*
             • skills        (pgvector     AKS-MCP
               at /skills    if enabled)   GitLab MCP
                              │            Git MCP
                              ▼
                     LLM via agentgateway
                    (Azure OpenAI via UAMI)
                              │
                              ▼
                     Diagnosis JSON
                        + save_memory(...)
                        + (optional) A2A → gitops-remediation
```

Output schema enforced at the workflow level — malformed responses rejected and retried once.

OUTPUT VIA TEAMS HITL

Severity router → Adaptive Card → callback

```
Triage complete
      │
      ▼
Severity router
      │
      ├─[critical / remediation needed]─▶️ HITL approval gate
      │                                          │
      │                                Teams Adaptive Card via Logic App
      │                                  • Approve → resume workflow
      │                                  • Reject  → ticket only
      │                                  • Edit    → re-prompt agent
      │                                          │
      │                                Istio VirtualService + AuthZ Policy
      │                                  (HMAC token verify)
      │                                          │
      │                                Argo Events webhook EventSource
      │                                  → Sensor resumes workflow
      │
      ├─[warnings — best effort]
      │
      └────▶️ Ticket creation step
                   ├─ GitLab MCP → issue
                   ├─ ServiceNow MCP → incident (planned)
                   └─ Azure DevOps MCP → work item (planned)
```


Read-only triage bypasses the gate · only writes/remediation suspend awaiting human nod

TICKETING INTEGRATIONS

Where triage lands

INTEGRATION	STATUS	MCP SERVER	USE CASE
GitLab	PRODUCTION	`gitlab-mcp-server`	Default ticket destination · round-robin assignment
Microsoft Teams	PRODUCTION	Logic App webhook	Adaptive Card notifications · HITL surface
Mattermost	PRODUCTION	Webhook	Real-time channel
ServiceNow	DESIGNED	`servicenow-mcp` (planned)	Enterprise incident · severity mapping · escalation chains
Azure DevOps	DESIGNED	`ado-mcp` (planned)	Work item creation in Boards
PagerDuty	DESIGNED	`pagerduty-mcp` (planned)	Page on-call for P1 / P2

The incident-agent selects target(s) by severity + namespace ownership · one alert can spawn multiple downstream artefacts.

SECTION 03

Component Status

Six pillars · where each one is

HUMAN-IN-THE-LOOP (HITL)

Production today · Bot Framework on roadmap

ELEMENT	STATUS	LOCATION
Teams Adaptive Card payload format	PRODUCTION	Logic App
Argo Events webhook EventSource	SMOKE-TESTED	`test-approval-workflow.yaml`
Istio VirtualService routing the callback	PRODUCTION	Commit `f5fbdb1`
Istio AuthorizationPolicy (HMAC check)	PRODUCTION	Commit `f5fbdb1`
Mock bot for end-to-end test	PRODUCTION	`smoke-test.sh` (5-min validation)
Workflow `Suspend` template tied to callback	DESIGNED	Pattern doc only
Real Teams bot (replacing Logic App)	FUTURE	Bot Framework path documented

Why Logic App today, not Bot Framework: no app registration overhead. The cluster-side contract stays the same — only the Adaptive Card POST destination changes when we cut over.

AGENT-TO-AGENT (A2A) PROTOCOL

JSON-RPC 2.0 with sharp edges

WIRE FORMAT

```
POST /api/a2a/kagent/<agent>/
{
"jsonrpc": "2.0",
"id": "...",
"method": "message/send",
"params": {
"message": {
"role": "user",
"parts": [{"kind":"text","text":"..."}]
}
}
}
```

COMMON GOTCHAS

- Missing trailing slash → 404 silent
- Method `tasks/send` (old docs) → unsupported
- Parts missing `kind: "text"` → parse error
- Session API for sending → 403 even with right user

Agent-as-tool pattern

MANAGED LGTM INTEGRATION

Alloy is the only handle — bidirectional

DIRECTION	PATTERN	COMPONENT
Push metrics	ServiceMonitor / PodMonitor → remote_write	`prometheus.operator.*` → `prometheus.remote_write`
Push logs	Pod logs + K8s events	`loki.source.kubernetes` + `loki.source.kubernetes_events`
Push traces	OTLP receiver → Tempo	`otelcol.receiver.otlp` → `otelcol.exporter.otlp`
Provision rules	PrometheusRule + LokiRule CRDs → managed Ruler	`mimir.rules.kubernetes` + `loki.rules.kubernetes`
Loop alerts back	AM webhook → Kafka → Event Hub	`loki.source.api` → `otelcol.exporter.kafka`
Agent anomalies	Recording-rule baselines + z-score / cohort	LogQL loop detection
Triage prompt enrichment	Workflow queries Mimir/Loki at triage-time	Splices context into KAgent prompt

DESIGNED · Pending answers from platform team on Q1–Q14 · `aks-mgmt-stack/k8s-event-triage/managed-lgtm-integration/`

AGENTGATEWAY MIGRATION

LiteLLM → agentgateway · 3 phases

01

Parallel deploy

agentgateway alongside LiteLLM · second ModelConfig CRD · UAMI verified · per-agent migration in flight

CURRENT

02

Migrate all agents

Bulk patch ModelConfig refs across all kagent Agent CRDs · verify each · roll back if issue

NEXT

03

Remove LiteLLM

Delete deployment + ModelConfig · keep PostgreSQL for kagent memory when re-enabled

FUTURE

Why agentgateway: UAMI/workload identity native (no API keys to rotate) · MCP gateway · A2A gateway · OTel native · per-route guardrails.

SKILLS MECHANISM

Git-versioned, mounted via initContainer

```
Agent CRD spec.skills.gitRefs:
  - url: https://github.com/your-org/your-skills-repo.git
    ref: main
    path: skills/cert-diagnostics
    name: cert-diagnostics
                    │
                    ▼
        kagent injects "skills-init" initContainer
                    │
                    ▼
        Clones repos to emptyDir at /skills
                    │
                    ▼
        Main agent container mounts /skills read-only
        Discovers skills via SKILL.md frontmatter
        Executes scripts via built-in BashTool (sandboxed)
```

STATUS

Mechanism deployed and proven · library currently shallow

GROWTH

Driven by the learning loop — every triaged ticket closes only when an agent skill is updated

MEMORY STATUS

Designed, but blocked on kagent

⛔ Blocked on upstream kagent

kagent shipped `MemorySpec` in v0.8.0 then reverted in v0.8.3 · expected back in a future release.

ELEMENT	STATUS
PostgreSQL + pgvector backend	DEPLOYED on red cluster (currently used by LiteLLM)
pgvectorscale upgrade path	OPTIONAL · `timescale/timescaledb-ha` for StreamingDiskANN
Embedding ModelConfig	DESIGNED · separate from inference
`load_memory` / `save_memory` / `prefetch_memory` tools	BLOCKED · wait for kagent re-enable
Memory + A2A remediation design doc	DONE · `MEMORY-AND-A2A-REMEDIATION-DESIGN.md`

Workaround until re-enabled: GitLab issues are the long-term memory · workflow does fuzzy search of past tickets with same `event_reason` + `namespace` labels · top 3 spliced into agent prompt.

SECTION 04

Architecture Decisions

14 calls that shaped the design

DECISIONS 1 – 7

Specialisation, safety, contracts

#	DECISION	RATIONALE
1	Specialist agents per namespace, not generalist	Smaller models hallucinate when omniscient · focused prompts win
2	Read-only by default · write only via promotion	Blast radius is the hard control · RBAC-enforced not prompt-enforced
3	Namespace anchoring (`CRITICAL: use exact namespace "X"`)	Empirical fix for Qwen 14B typos · belt-and-braces alongside Kyverno
4	A2A as the inter-agent contract	Standardised JSON-RPC · kagent native · agentgateway-routable
5	No `kubectl edit` from agents — `patch` / `apply` only	Interactive editors via MCP break · `patch` is deterministic + auditable
6	GitOps remediation via MR, not direct cluster writes	Agent never `kubectl apply`s · opens MR · Flux applies · auditable, reversible
7	Triage agent ≠ remediation agent	Two CRDs, two SAs, two RBAC levels · never an upgrade in place

DECISIONS 8 – 14

Operations, gates, scaling

#	DECISION	RATIONALE
8	HITL gate before any write action	Approve/Reject/Edit · auto-graduates to no-gate after N successes
9	Worker-local triage where possible	Eliminates Event Hub round-trip for self-contained namespaces
10	Round-robin GitLab assignment, no namespace ownership	Builds breadth · visible at standups via Kanban
11	Skills are git-versioned, not in-line in CRDs	`gitRefs` · skills library reviewable in PRs · CRD stays small
12	Agent prompts committed to Git	`systemMessage` is a code artefact · changes go through PR review
13	3-layer dedup (Alloy / Sensor rate / script TTL)	Single layer brittle under storms · three layers means one can fail
14	agentgateway as single LLM exit point	One place for token tracking, guardrails, UAMI auth, OTel · agents don't know about Azure OpenAI directly

SECTION 05

Agent Cards

~20 agents · 5 grouped patterns

COMMON CARD FIELDS

Shared across all cards (unless flagged)

FIELD	COMMON VALUE
Process	Receive A2A `message/send` → load skills + memory → call MCP tools → summarise → return JSON-RPC result. Optional A2A call to `gitops-remediation-agent` for fixes.
Communication	Inbound: A2A from Argo Workflow. Outbound: A2A to other agents · MCP to `kagent-tool-server`, `aks-mcp`, `gitlab-mcp-server`, `git-mcp-server`.
Tools (read)	`k8s_get_resources` · `k8s_describe_resource` · `k8s_get_pod_logs` · `k8s_get_events` · `k8s_get_resource_yaml`
Tools (write — remediation tier only)	`k8s_patch_resource` · `k8s_apply_manifest` · `k8s_delete_resource` · `k8s_label_resource` · `k8s_annotate_resource` · `k8s_execute_command`
Knowledge	System prompt + `/skills` git bundle + (when enabled) pgvector memory + `kagent-builtin-prompts` ConfigMap
Evaluation	Output schema enforced (Issue/Affected/Root Cause/Remediation/Risk/Verification) · workflow grades fields and rejects malformed
Security	Read-only RBAC default · write via explicit RoleBinding to specific namespaces · UAMI scope per cluster · prompt constraints
Model routing	All → agentgateway → Azure OpenAI (UAMI) or local vLLM · same `default-model-config`

CARD A — TRIAGE SPECIALIST

The dominant pattern (covers ~20 agents)

PROCESS

Reactive — invoked by workflow on K8s Warning event. Optionally proactive via CronWorkflow (compliance, cost, security).

USER PERSONA

SRE / Platform Engineer responding to a triage notification — wants structured diagnosis + ranked remediation + exact kubectl commands.

PROBLEM

Domain-specific K8s failures hit a namespace · on-call lacks deep specialist knowledge · agent collapses 30–60 min into seconds.

ROLE

Read-only investigator + remediation suggester. Writes nothing. Returns analysis JSON to the workflow.

Applies to: cert-manager · kyverno · kro · external-secrets · reloader · flux-system · gatekeeper-system · istio-ingress · istio-system · kube-system · dns · database · observability · cost · change · network · storage · infra · security · compliance.

CARD A — SPECIALIST SKILLS

What lives in `/skills/<agent>/`

AGENT	SKILLS BUNDLE
`cert-manager-agent`	`cert-diagnostics` · ACME challenge inspection · issuer status · expiry scans
`network-agent`	`cni-health-probe` · `ingress-backend-walker` · `istio-config-analyse`
`storage-agent`	`pvc-binding-trace` · `longhorn-volume-health` · `csi-driver-probe`
`infra-agent`	`node-condition-decoder` · `kubelet-log-grep` · `inotify-pressure-check`
`security-agent`	`pss-violation-decoder` · `falco-event-correlator` · `rbac-audit`
`cost-agent`	`right-size-recommender` · `idle-resource-finder` · `node-utilisation-report`
`compliance-agent`	`label-compliance-scan` · `quota-coverage-scan` · `network-policy-coverage`
`observability-agent`	`prometheus-target-health` · `loki-stream-audit` · `cardinality-explorer`

Full per-agent definitions: `aks-mgmt-stack/k8s-event-triage/AGENT-ROSTER.md`

CARD B — REMEDIATION AGENT

Limited write · gated by HITL approval

PROCESS

Invoked only after HITL approval. Receives diagnosis + approval token. Executes recommended kubectl actions in a controlled subset.

ALLOWED ACTIONS

`patch` · `scale` · `restart` · `label/annotate`

FORBIDDEN

Delete CRDs · delete PVCs with data · restart all replicas at once · anything outside approved namespace

SECURITY

Approval token verified against workflow HMAC · replay protection · action whitelist enforced server-side · failures escalate, never retry

Applies to: `sre-remediation-agent`. All writes go through GitOps where possible · direct cluster writes only for break-glass.

CARD C — GITOPS REMEDIATION AGENT

Branch + commit + MR · never `kubectl apply`

PROCESS

Receives A2A call from triage agent describing needed config change. Clones GitOps repo · branch · diff · commit · MR. Never pushes to main.

USER PERSONA

SRE reviewing the MR — wants clean single-purpose diff with agent diagnosis as MR description and GitLab triage issue linked.

TOOLS

`git-mcp-server` (clone/checkout/commit/push/diff/file_*) · `gitlab-mcp-server` with `requireApproval` on `create_merge_request`

STATUS

DESIGNED · needs git-mcp-server + gitlab-mcp-server · proven A2A pattern (Kimi 6/6)

Bot account scoped to single GitOps project · MR title `[Auto-Fix]` · one change per MR · GPG-signed · detect-secrets pre-commit.

CARD D — SYSTEM AGENTS (MGMT CLUSTER)

Cluster-wide read · scheduled CronWorkflow

AGENT	SPECIALIST SKILLS	MOSTLY
`incident-agent`	`incident-correlator` · `pagerduty-escalator` · `mttr-calculator`	Process management
`change-agent`	`rollout-watcher` · `canary-comparator` · `argocd-sync-checker`	Deployment health
`compliance-agent`	`weekly-compliance-report` · `audit-evidence-exporter`	Reports + scorecards
`cost-agent`	`weekly-cost-report` · `right-sizing-recommender`	FinOps reports
`observability-agent`	`monitoring-stack-health` · `cardinality-explosion-detector`	Stack-of-stacks monitoring

User persona: Engineering leadership · FinOps · SRE management. Outputs: weekly reports as GitLab issues · weekly Teams summaries · PagerDuty pages (incident-agent only). RBAC: read across all namespaces · incident-agent has write access to ticketing systems only.

CARD E — COORDINATOR (FUTURE)

Decompose multi-domain incidents

PROBLEM

A TLS outage spans cert-manager + DNS + ingress · today produces three separate triages · coordinator joins them into one narrative.

ROLE

Pure orchestration · holds no domain knowledge · calls specialists (parallel or sequential with revision loop) · synthesises.

TOOLS

None of its own · only the list of available agents (auto-discovered via kagent registry / A2A skill metadata).

RISK CONTROL

`max_parallel_agents` + per-coordinator token budget in agentgateway · prevents fan-out storm

PLANNED · pattern proven (Kimi 6/6 dev pipeline test) · not yet wired into triage system.

SECTION 06

Open Work + Risks

What's pending and where it could bite

PENDING ITEMS

What's open · who owns it · what's blocking

ITEM	STATUS	OWNER	BLOCKER
LGTM integration deployment	DESIGNED	Platform team Q&A	OPEN-QUESTIONS Q1–Q14
kagent memory re-enable	PENDING	kagent upstream	v0.8.3 reverted · awaiting next release
`gitops-remediation-agent` build	NOT STARTED	David	needs git-mcp + gitlab-mcp servers
ServiceNow MCP server	NOT STARTED	TBD	Vendor evaluation
Azure DevOps MCP server	NOT STARTED	TBD	Vendor evaluation
Real Teams Bot (replace Logic App)	DESIGNED	TBD	App registration + Bot Framework deploy
LiteLLM → agentgateway full migration	PHASE 1 OF 3	David	Per-agent ModelConfig migration
Skills library expansion	CONTINUOUS	All engineers	Driven by learning loop
Specialist agents on workers	11 OF ~20	David → SRE	Phase 3 of `ENGINEERING-ROLLOUT.md`
Round-robin GitLab assignment	DESIGNED	David	Needs dedicated GitLab project + roster ConfigMap

TOP 5 RISKS

Where we'd most likely get bitten

RISK 01

Memory pending kagent re-enable

Without it, agents re-investigate from scratch each fire · GitLab-issue-as-memory is lower fidelity

RISK 02

agentgateway migration not finished

Running both proxies in parallel doubles operational surface · cutover discipline matters

RISK 03

HITL via Logic App

No per-user authorisation · audit trail thin beyond Run History · Bot Framework migration would tighten

RISK 04

Skills library shallow

Learning loop is the only growth mechanism · if engineers don't update skills when closing tickets, agents don't get smarter

RISK 05

Cardinality budget on managed LGTM

`gen_ai_request_model × agent × cluster × tenant` is a lot of label combinations once we hit 20 agents · platform-team budget unknown until Q7 answered

DOCUMENT MAP

Where to go for the detail

DOCUMENT	PURPOSE
`docs/CDA-DESIGN-AUTHORITY.md` (v1.1)	Formal design authority position
`docs/PROJECT-STATUS.md`	Long-form companion to this deck
`docs/CDA-SLIDES.md` (v1.1)	CDA as slides
`docs/SAD-THREAT-MODEL.md`	STRIDE threat model
`docs/SAD-LOGGING-MONITORING-AUTH.md`	Logging/monitoring/auth
`docs/SAD-LOGGING-MONITORING-LLM.md`	LLM governance
`worker-cluster-bundle/SKILLS-AND-REMEDIATION.md`	Skills loading mechanism
`worker-cluster-bundle/MEMORY-AND-A2A-REMEDIATION-DESIGN.md`	Memory + A2A remediation design
`worker-cluster-bundle/AGENTGATEWAY-TRANSITION.md`	LiteLLM → agentgateway migration
`aks-mgmt-stack/k8s-event-triage/managed-lgtm-integration/`	Managed LGTM design + agent anomalies
`aks-mgmt-stack/k8s-event-triage/AGENT-ROSTER.md`	Full per-agent specifications

THANK YOU

Questions?

Platform Engineering · David Gardiner

Companion document: `docs/PROJECT-STATUS.md`

Render: `marp PROJECT-STATUS-SLIDES.md --pdf --allow-local-files`